Munich University of Applied Sciences
HM Business School

From Vibe Coding to Production-Ready Software

Developing and Evaluating a Governance and Development Framework for Generative AI-Assisted Software Creation by Non-Technical Employees

Master's Thesis

Submitted by:
Nassib Aboud

Degree Program:
Business Information Systems (M.Sc.)

Matriculation Number:
34140924

Contact:
nassib.aboud@hm.edu

First Examiner:
Prof. Dr. Markus Thimmel

Second Examiner:
Prof. Dr. Christoph Ertl

Submission Date:
30 October 2026

Declaration on the Use of Generative AI

Generative artificial intelligence tools were used as supporting tools during the preparation of this Master's thesis.

Their use included, where applicable, support with literature discovery, brainstorming and structuring, language and style improvement, critical review of written material, and assistance with software development and coding tasks conducted as part of the research.

Generative AI outputs were not treated as independent academic sources. Academic claims and evidence were verified against the original sources where applicable.

AI-generated suggestions, analyses, text, and code were critically reviewed, selected, modified, or rejected by the author.

The author remains fully responsible for the research design, methodological decisions, interpretation of results, argumentation, conclusions, and final content of this thesis.

The use of generative AI tools was conducted in accordance with the applicable requirements of Munich University of Applied Sciences and the principles of good scientific practice.

Declaration of Originality

I hereby declare that I have prepared this Master's thesis independently and that I have used only the sources, materials, and tools indicated in the thesis.

All passages that have been taken directly or indirectly from published or unpublished sources have been identified accordingly.

The use of generative artificial intelligence and other digital tools has been disclosed where required.

I further declare that this thesis has not previously been submitted, in the same or substantially similar form, as part of another examination procedure.

Munich, 6 September 2026

Nassib Aboud

Abstract

Insert abstract here.

Contents

Abstract **3**

1 Introduction **6**

1.1 Research Objectives and Research Questions 6

1.2 Scope and Delimitations 6

2 Theoretical Background and Related Research **6**

2.1 Conceptual Foundations and Key Definitions 6

2.2 Generative AI-Assisted Development in software development 8

2.3 Capabilities and Limitations of Non-Technical Developers 8

2.4 Expert Support in Citizen-Led Software Development 8

2.5 Governance, Guardrails, and Development Support 8

2.6 Conceptualisation of Production Readiness 8

2.7 Synthesis of the Literature and Research Gap 8

3 Research Methodology **8**

3.1 Research Approach and Methodological Rationale 9

3.2 Research Process 9

3.3 Research Setting and Researcher Role 9

3.4 Participant and Use-Case Selection 9

3.5 Data Collection 9

3.6 Data Analysis 9

3.7 Operationalisation of Production Readiness 9

3.8 Framework Evaluation Strategy 9

3.9 Research Quality, Ethics, and Data Protection 9

4 The Governance and Development Framework **9**

4.1 The Enabling Environment 9

4.1.1 Technology Platform 9

4.1.2 Governance Structure 9

4.2 Working Practices 9

4.2.1 Core Concepts and Principles 9

4.2.2 The Stage-Gate Process 9

4.2.2.1 Prototype Exploration Stage 9

4.2.2.2 Production Development Stage 11

4.2.2.2.1 General Best Practices 11

4.2.2.3 The Agile V Development Framework 12

4.2.2.3.1 Left side: specifying the change 13

4.2.2.3.2 Bottom: building the change 15

4.2.2.3.3 Right side: verifying and accepting the change 18
4.2.2.3.4 Testing and verification 19

5 Evaluation and Findings **19**

6 Discussion **19**

7 Conclusion **19**

1 Introduction

1.1 Research Objectives and Research Questions

1.2 Scope and Delimitations

2 Theoretical Background and Related Research

2.1 Conceptual Foundations and Key Definitions

AI-assisted, agentic, and vibe coding. AI-supported software development today is not one single thing. It runs along a spectrum defined by degrees of AI autonomy, interaction modalities, and human oversight (Farrag, 2026; Mandl & Mandl, 2026; Sapkota et al., 2025). Grounded in the AI-Augmented Methodology Taxonomy (AAMT) proposed by Farrag (2026), this spectrum can be categorized into three distinct tiers:

1. **Tier 1: Passive Suggestion (AI-Assisted Coding):** At this baseline, Generative AI operates as an inline autocompletion engine within the developer's Integrated Development Environment (IDE) (Díaz et al., 2026; Farrag, 2026). The AI assists with localized tasks such as filling short code blocks, suggesting syntax, or drafting basic unit tests. The human developer remains the primary code author, maintaining direct cognitive engagement and line-by-line inspection (Díaz et al., 2026).
2. **Tier 2: Active Generation (Vibe Coding):** Coined initially by Karpathy and formalized by Meske et al. (2025), *vibe coding* shifts software creation from deterministic syntax authoring to probabilistic *intent mediation*. Through multi-turn natural language dialogue, developers articulate high-level goals ("the vibe"), while the AI generates entire components or scripts (Farrag, 2026; Meske et al., 2025). In this mode, the primary human role shifts from direct implementation to conversational orchestration and output review (Díaz et al., 2026; Sapkota et al., 2025).
3. **Tier 3: Autonomous Agency (Agentic Coding):** Positioned at the autonomous end of the spectrum, Agentic Software Engineering (ASE) delegates goal-level technical objectives (e.g., refactoring a legacy service or implementing an end-to-end API) to autonomous multi-agent systems (Díaz et al., 2026; Sapkota et al., 2025). Operating within sandboxed execution environments, agentic systems independently formulate execution plans, modify repository files, invoke terminal commands, run test suites, and repair failing code with minimal step-by-step human intervention (Mandl & Mandl, 2026; Sapkota et al., 2025).

The labels used for AI-supported software development are not yet stable. Beyond "vibe coding" and "agentic coding", the literature uses several partly overlapping terms for systems that plan and carry out software tasks with limited human intervention. These include "Agentic Software Engineering" or "SE 3.0" (Hassan et al., 2025), "AI teammates" (Hassan et al., 2024; Li et al., 2025), "autonomous coding agents" (Li et al., 2025), and "AI software engineer" or "agentic software engineer" (Hoda, 2025). These terms differ in emphasis rather than in the underlying phenomenon. Some describe a tool (*coding agent*), some a role (*AI teammate*, *AI software engineer*), and some a discipline or era (*Agentic Software Engineering*, *SE 3.0*). Hassan et al. (2025), for instance, distinguish one-to-one "agentic coding" from team-level "agentic software engineering". Hoda (2025) observe that terminology drift is already visible even though the field is nascent. Even the term "vibe coding" is defined inconsistently. Sapkota et al. (2025) frame it as a human-in-the-loop counterpart to agentic coding, whereas Ge et al. (2025) define it as the practice of managing coding agents. Because this vocabulary is still evolving and partly shaped by vendor marketing, this thesis does not adopt any single label as authoritative. Instead, it uses the terms defined in Section X as working definitions and characterises each use case by its degree of AI autonomy and human engagement with the code, not by its label.

On one end sits AI-assisted coding, where AI works as a small helper inside the developer's editor (IDE). It suggests the next line of code, fills in short blocks, writes basic tests, or drafts documentation. The developer still writes and reads the code. On the other end sits agentic coding. Here, the developer gives the AI a high-level goal—for example, "migrate this repository from Python 2 to Python 3"—and the AI

plans the steps, opens files, runs tests in a safe sandbox, reads the error logs, and fixes itself with little human help (Galster et al., 2026; Sapkota et al., 2025). Between and around these two sits vibe coding (VC), a term coined by Andrej Karpathy in February 2025 (Karpathy, 2025). Sarkar and Drosos (2025) describe vibe coding as "material disengagement": the programmer no longer works directly on the code itself but steers the AI through natural-language chat, screenshots, and quick "vibe checks" on whether the running app feels right.

From chat window to terminal: YOLO, Claude Code, Codex, and MCP. For a long time, using an LLM for coding meant copy-and-paste: the developer copied code into a chat window, got a suggestion, and pasted it back. Newer tools remove this manual loop. Claude Code and OpenAI Codex CLI run directly in the terminal. They can read the whole repository, edit files, run shell commands, and execute tests on their own (Galster et al., 2026). This is often used in "YOLO mode" — the developer accepts all changes without reading each diff, trusting the agent to move forward. On top of this, the Model Context Protocol (MCP) lets the agent connect to external tools and data sources (databases, ticket systems, documentation, other MCP servers) through a standard interface (Galster et al., 2026). In parallel, sub-agents can be spawned for specific jobs (e.g., a reviewer agent, a test-writer agent) and only their final output flows back to the main agent (Galster et al., 2026). This is a real historical shift: from "suggest a snippet in a chat window" to "work autonomously inside my codebase with access to my tools".

Deterministic tools vs. probabilistic foundation models. To understand why this shift matters, it helps to look at what came before. In classical software engineering — and in low-code / no-code (LCNC) platforms — the tools are deterministic. This means the same input always gives the same output, following fixed, hard-coded rules. LCNC platforms use Model-Driven Development (MDD): a citizen developer drags visual blocks onto a canvas to draw the business logic, and the platform translates this visual model into runnable code or database configurations through strict compiler-like rules (François et al., 2025; Gadde, 2025). There is no guesswork. The compiler acts as a strict guardrail: it does not let the user produce an invalid SQL statement or an unclosed bracket.

Generative AI works differently. Large Language Models (LLMs) are probabilistic. They do not "know" syntax the way a compiler does. Instead, they use autoregressive next-token prediction: for every next word (token) in the output, they compute the statistical probability of what comes next, based on patterns learned from very large training corpora of code and text (Mandl & Mandl, 2026). Two consequences follow. First, the same natural-language prompt can produce different code on different runs — the model is non-deterministic by design. Second, because the output is what is statistically plausible and not what is mathematically verified, LLMs often produce code that looks clean and professional but contains hallucinated libraries, security holes, or architectural inconsistencies (Farrag, 2026).

Why this matters for the SIXT thesis: the Productivity–Reliability Paradox. This deterministic vs. probabilistic split is the root cause of the Productivity–Reliability Paradox (PRP) described by Farrag (2026). Low-code has built-in guardrails and is therefore safe but slow and narrow. Vibe coding is fast and open but has no built-in guardrails; if the developer blindly accepts every suggestion, the codebase quickly fills up with hidden or "shadow" technical debt (Farrag, 2026; Kobiella et al., 2026). The idea of this thesis is to bridge the two: to take a very fast but volatile probabilistic engine (vibe coding) and wrap it inside a strict deterministic harness — CI/CD pipelines, automated security tests, and expert stage-gates — so that all AI-generated code must pass through fixed, machine-checked rules before it reaches production. In the terms of Galster et al. (2026), this is harness engineering: configuring the environment around the model (context files, skills, sub-agents, MCP tools, hooks) so the model can move fast without breaking the rules of the enterprise. The goal is simple: the speed of vibe coding, with the safety of classic software engineering.

2.2 Generative AI-Assisted Development in software development

2.3 Capabilities and Limitations of Non-Technical Developers

2.4 Expert Support in Citizen-Led Software Development

2.5 Governance, Guardrails, and Development Support

2.6 Conceptualisation of Production Readiness

2.7 Synthesis of the Literature and Research Gap

3 Research Methodology

Fetzer et al. (2026) demonstrate how Action Design Research can guide the responsible engineering of generative AI systems in an organizational setting.

3.1 Research Approach and Methodological Rationale

3.2 Research Process

3.3 Research Setting and Researcher Role

3.4 Participant and Use-Case Selection

3.5 Data Collection

3.6 Data Analysis

3.7 Operationalisation of Production Readiness

3.8 Framework Evaluation Strategy

3.9 Research Quality, Ethics, and Data Protection

4 The Governance and Development Framework

4.1 The Enabling Environment

4.1.1 Technology Platform

4.1.2 Governance Structure

4.2 Working Practices

4.2.1 Core Concepts and Principles

4.2.2 The Stage-Gate Process

4.2.2.1 Prototype Exploration Stage

The framework uses a lightweight, V-shaped exploration cycle during prototype development and evaluation. It selectively adapts the connection between requirements and evaluation found in Agile V (Koch & Wellbrock, 2026). It is a design adaptation of this thesis, rather than the complete Agile-V process. The SCOPE-V task cycle, separate verification agents and detailed engineering evidence are reserved for the subsequent production-development stage, as they are not needed at this stage and would only create extra work. The Agile-V framework was chosen because AI-assisted development shifts the main challenge from generating code to verifying that the resulting software meets business requirements, functions reliably and satisfies quality standards. Its main advantage is the direct connection between requirements and verification, ensuring that each development decision can be evaluated against predefined expectations rather than relying on plausible AI-generated outputs. Combining the V-Model's structured verification with Agile's iterative flexibility makes it suitable for both development stages: during prototyping, a lightweight V-cycle supports rapid exploration, feedback and validation of business needs; during production development, more rigorous task-level V-cycles introduce independent testing, traceability and documented evidence. This allows the same underlying principle—verification before acceptance—to guide both stages while adapting the level of control to their different objectives and risks.

Koch (2026) propose converting exploratory dialogues with different LLMs into a reviewed execution brief before starting the develeopment (in this case, the prototype creation), because long conversations can contain superseded assumptions and implicit constraints. This thesis operationalises that principle through a specification handover into a fresh chat or project context before building the consolidated prototype. Similar practices have been observed in vibe coding: developers deliberately opened a new

chat or cleared the AI's context before starting a new phase of work (Sarkar & Drosos, 2025), and non-technical participants used a general-purpose chatbot to turn their ideas into a structured prompt or prototype description for a prototyping tool (Kobiella et al., 2026).

The prototype may consist of screens, a clickable mock-up or a limited interactive demonstration created with an AI-assisted design or prototyping tool. Its purpose is to clarify the business need, explore the proposed workflow and gather early user feedback before committing to production engineering.

This step of prototyping is especially critical for non-technical citizen developers, who face distinct communicative and evaluative barriers when building software with AI. In a controlled study with 67 non-programmers, participants struggled to express their intent in natural language and lacked the technical vocabulary that code-generating models respond to (Feldman & Anderson, 2024). In a case study, a person without coding experience could not produce a working driving simulator with a single prompt. Features that combined several systems required many iterations, and vague prompts led to unstable results (Fortes-Ferreira et al., 2025). Even experienced programmers found it difficult to describe visual ideas, such as diagrams, in words (Sarkar & Drosos, 2025). Conversely, non-programmers assisted by a conversational AI performed comparatively well when tasks were small and fully specified with business rules and test cases (Esnaola et al., 2026).

A prototype addresses these barriers. Instead of describing elements such as navigation, filters or button states in words alone, the citizen developer can show, inspect and correct them step by step. Kobiella et al. (2026) found that AI prototyping tools enabled non-technical users to create high-fidelity prototypes for exploration and stakeholder communication. The same study shows why this form of review suits citizen developers. Non-technical participants could judge whether the output looked as intended, but not whether the underlying code was good, performant or secure (Kobiella et al., 2026). Prototyping therefore relies on a form of validation that citizen developers can actually perform.

Furthermore, a clearly defined prototype is likely to reduce changes during production development. Participants in Kobiella et al. (2026) contrasted the relative ease of building and adding features with the difficulty of modifying or removing them. People with basic coding knowledge could make minor adjustments directly in the code, but non-technical users struggled. One participant noted that without coding experience, even small layout changes would require "even more AI calls" (Kobiella et al., 2026). They also gained little from reading generated code or error messages and when their attempts to fix or improve the code using LLMs failed, many ran out of strategies for what to try next (Feldman & Anderson, 2024). Non-technical users also reported that moving from a prototype to a deployed, working product was very challenging (Kobiella et al., 2026). Clarifying what should be built before production development therefore matters more for citizen developers, because later corrections are harder for them to make. This difficulty can be explained through two possible development patterns: compounding error and compounding correctness. These terms, which are common in the practice of AI-assisted development (StrongDM AI, 2026), are used here as a synthesis of sources rather than as established or quantitatively validated laws (see Figure 2).

For the proposed framework, these ideas support placing a prototyping step between the idea and production development especially for citizen developers. During prototyping, the specification and user walkthroughs help identify misunderstandings about what should be built and should reduce rework and improve outcomes for citizen developers. This will be evaluated through eADR.

Since AI-generated code can appear correct while containing hidden logical errors Wang (2026), we adopt a V-shaped framework in which requirements are defined on the left, the prototype is built at the bottom, and its behaviour is systematically tested against those requirements on the right (see Figure 1).

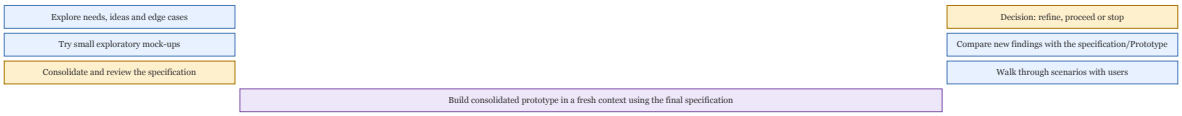


Figure 1. Own representation of a V-shaped prototype exploration cycle, inspired by the requirement–evaluation relationship in Agile V (Koch & Wellbrock, 2026).

On the left side, the citizen developer explores the business problem with an AI assistant, such as ChatGPT, and relevant stakeholders. The feature set does not need to be known in advance. Exploration may involve discussing requirements, questioning assumptions, identifying edge cases and generating

alternative designs. Small, disposable mock-ups may be created to investigate individual screens or interactions. These exploratory outputs help clarify the solution but are not automatically treated as agreed requirements.

Before building the consolidated prototype, the citizen developer converts the findings into a concise, standalone specification. It identifies the intended users, selected features, main user journeys, relevant edge cases, expected outcomes, exclusions and any simulated behaviour. Contradictory or outdated ideas are removed, and unresolved questions are recorded explicitly. The citizen developer reviews the specification for consistency and business relevance. A more experienced colleague or relevant specialist is involved where uncertainty warrants it.

At the bottom of the V, the consolidated prototype is built in a fresh chat or project context using the reviewed specification and deliberately selected reference material. A tool such as Claude Design, Figma or another prototyping environment may support this activity. The exploratory conversation is not carried forward as the authoritative build instruction. Instead, the specification provides a clear baseline for what should be demonstrated. The work remains within the approved experiment boundaries, including permitted tools and data.

On the right side, the citizen developer walks through the specified scenarios with intended users or relevant business stakeholders. Findings are compared with the expected outcomes, and a short record captures useful elements, shortcomings and unanswered questions. If feedback changes the intended behaviour, the specification is updated before the prototype is revised. Minor corrections can continue within the build context. Substantial changes return to specification review and may justify a new build context. These return paths are omitted from the figure for readability.

4.2.2.2 Production Development Stage

4.2.2.2.1 General Best Practices

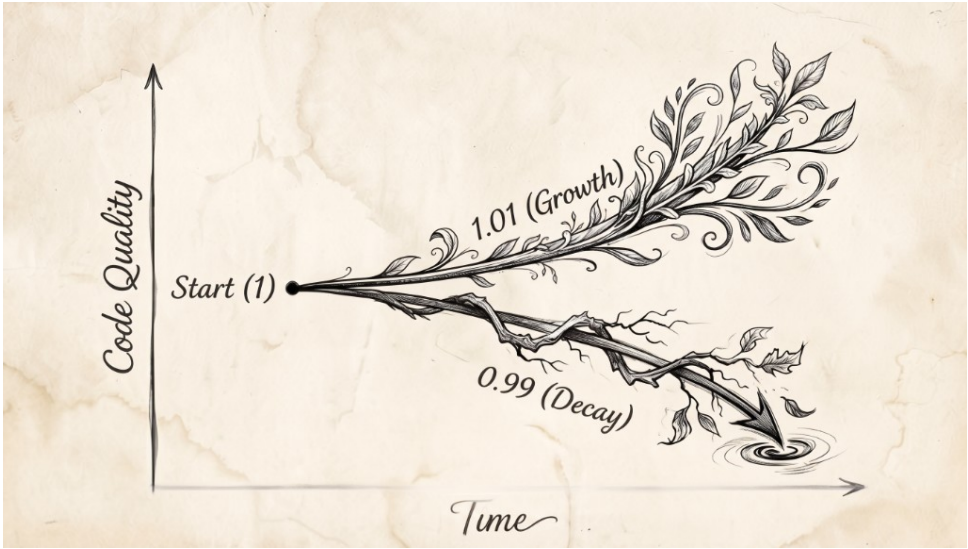


Figure 2. Conceptual illustration of compounding correctness and compounding error in AI-assisted development (own illustration).

Without structural guardrails, unstructured vibe coding can create a negative feedback loop. Initial defects, unstated assumptions and hallucinated logic may be carried forward and build on each other across multi-turn interactions, making subsequent changes harder. Sarkar and Drosos (2025) describe "context momentum": earlier prompts and generated outputs influence later development and can make it difficult to move away from initial implementation choices. In their example, the model interpreted a request for historical exchange rates as a single date. When the developer later explicitly requested a date range, the newly generated code still fetched only one date. If such choices are made directly in the production build, they may become fixed before the requirements are fully understood. A related concern is epistemological debt, which Ginac (n.d.) defines as the gap between a system's behaviour and its

maintainers' understanding of it. This conceptual framework suggests that accepting generated changes without maintaining that understanding can weaken the ability to diagnose and correct failures. It does not establish that errors inevitably multiply with every interaction. Prototyping can partly address both concerns: implementation choices are explored before the production build, and citizen developers can understand the intended behaviour before production code is generated.

When vibe coding is bounded by a technical harness, explicit specifications and verification against ground truth, later iterations can build on validated results. Correctness and quality may then accumulate over time. A technical harness is a configured environment of tools, constraints and automated checks. A validated prototype allows production development to start from a reviewed basis. When the specification derived from the prototype is reviewed, changes are checked against explicit expected behaviour, and validated corrections are retained in tests and rules, later iterations can build on earlier improvements. In an industry experience report, Lopopolo (2026) describes using enforceable architectural constraints, repository documentation and automated feedback to guide coding agents. Similarly, Wang (2026) proposes developer-validated contracts that connect intended behaviour with code generation, testing and verification. VibeContract is a vision paper supported by an illustrative example, rather than broad empirical validation. In the proposed framework, the validated prototype is the source of this intended behaviour. Together, these sources motivate the possibility of cumulative quality improvements. They do not guarantee that every iteration increases correctness.

4.2.2.3 The Agile V Development Framework

In the production-development stage, the framework uses a more complete version of Agile V than in the prototype stage. Each feature or larger change now goes through its own small V-cycle. This cycle includes independent testing, traceability (a clear link between each requirement and its tests) and documented evidence. It combines two elements from earlier work. From Agile V, it takes the idea that separate AI agents build the software, write the tests, run the tests and collect the evidence (Koch & Wellbrock, 2026). From Agentic Agile-V, it takes the six-step SCOPE-V task cycle (Koch, 2026). These are complemented by spec-driven development, contracts and practitioner approaches to planning, which are introduced below. How these elements are combined, assigned to agents and placed within the stage-gate lifecycle is a design choice of this thesis. It is important to note that the framework builds upon the technical and governance setup provided by the company explained in Section TBD!!.

The cycle is based on three principles. The first is spec-driven development (SDD). Every meaningful change starts from a written specification that is stored and versioned together with the code. This specification acts as the contract between people and AI agents. Code, tests and documentation are derived from it. If the specification and the code disagree, the specification counts as correct (Díaz et al., 2026). Furthermore, the specification documentation are complemented by a shared overall plan for the overall change of the agile V cycle (Lopopolo, 2026). The plan, kept in a file such as plan.md, is created by a Planning Agent from the specification once it has been approved at Checkpoint A. The third principle is contract- and test-based verification. A change is accepted because independent tests and other evidence show that it matches the specification, not because the result looks convincing (Koch, 2026). The V-shape links the principles: the left side creates the specification, the bottom builds the change, and the right side checks that the change matches the specification and all the updates are then saved in the plan.md. Most sources behind these ideas are conceptual or based on practitioner experience, and they have not been tested with non-technical users. Their suitability is therefore evaluated through eADR.

As shown in the prototype stage, non-technical participants can judge whether software looks and behaves as intended. A stricter structure is needed at this stage because what has to be checked changes. They cannot judge whether the code behind it is correct, secure or easy to maintain. In other words, citizen developers can mainly check functional requirements, that is, what the software does. Especially with specification-driven development (SDD), a clear specification allows reviewers to check whether the evidence demonstrates that the requirements have been met, instead of trying to work out the intended behaviour from the code (Díaz et al., 2026). Still most non-functional requirements, such as security, data protection, reliability and maintainability, are hidden in the code and lie beyond their judgement. Checking these qualities must therefore be shifted as far as possible to other means: the technical setup provided by the company and department, such as approved platforms and automated checks; AI agents that test and analyse the code; and, where needed, supporting stakeholders such as IT, information security or data protection.

To support the with non-functional requirements, Agile V gives two different jobs to two different agents. A Test Designer writes tests based only on the requirements and never sees the code, while a separate Build Agent writes the code. This separation is meant to prevent tests that simply confirm what the code happens to do instead of what it should do (Koch & Wellbrock, 2026). Confirmed behaviour is kept in tests that every later change must also pass. In this way, quality can build up over time, which is how the framework aims at compounding correctness (see Figure 2).

The cycle operates on three nested levels. In Agentic Agile-V, Agile V steers the project as a whole, while SCOPE-V steers the individual tasks carried out by agents (Koch, 2026). In this thesis, the stage-gate lifecycle takes over the steering of the application as a whole, including investment, ownership, pilot and release. Agile V moves one level down and steers how each single change is specified, built and accepted during the engineering and verification stage. SCOPE-V remains at the lowest level and steers how the Build Agent carries out each small task within a change. For example, in an HR leave-request tool, the stage-gate lifecycle decides whether the tool may be piloted, a V-cycle handles the feature for submitting a request, and SCOPE-V handles single tasks within it, such as ensuring that the leave balance cannot become negative. A V-cycle therefore ends when a tested change is accepted, not when the application is released; the release remains a decision of the stage-gate lifecycle (see Figure 3).

4.2.2.3.1 Left side: specifying the change

Following SDD, the specification is divided into two levels with different purposes (Díaz et al., 2026). The system specification describes the stable environment of the application: architecture rules, coding standards and other rules that every change must respect. It is stored in a rule file, such as AGENTS.md, which the agents read at the start of every session. The feature specification describes only the change currently being built within the Agile V cycle, including its acceptance criteria. Díaz et al. (2026) compare these two levels to the difference between organisational policy and operational work instructions. Keeping them apart matters: if an agent only receives the feature specification, it may produce code that works but ignores the general rules; if it only receives the system specification, it does not know what to build (Díaz et al., 2026). In their model, system specifications are typically owned by technical leads and architects, whereas feature specifications are owned by the developer responsible for the change. This thesis adapts this division to citizen development: IT maintains the non-functional system specification, while citizen developers write the feature specification for their functional changes. As a result, citizen developers only need to describe their business need, while technical and data-protection rules, defined once by IT, automatically apply to every change. Some non-functional requirements, however, depend on the specific feature and its business context and are therefore part of the feature specification.

The citizen developer is responsible for the feature specification. Starting from the tested prototype and its specification, they describe the business need, the intended users and the expected behaviour. Requirements are written as acceptance criteria in the form Given–When–Then, so that each criterion can be turned directly into a test (Díaz et al., 2026). Regarding the non-functional requiements specific to the feature, AI-generated requirements often leave out non-functional aspects. Quality requirements such as response time, security or the handling of personal data are therefore written down as short quality scenarios with a measurable target (**waseem2025**). In addition, task-level contracts make hidden business rules explicit (Wang, 2026). Preconditions state what must be true before an action, postconditions what must be true after it, and invariants what must always remain true, for example that a remaining leave balance can never be negative. An AI assistant can draft these criteria, scenarios and contracts from the prototype specification. The citizen developer then checks them, because they describe business rules and expectations rather than technical details. Technical thresholds are confirmed with IT where needed. The resulting brief also states the goal, what is out of scope, which parts of the application are affected, which evidence is required and the risk class (Koch, 2026).

At Checkpoint A, the scope, constraints and risk class are confirmed. Checkpoint A therefore works as the conversation-to-contract gate proposed by Koch (2026): agents build from the reviewed specification, not from the earlier chat. So that citizen developers do not approve their own risk level, the risk class is set by predefined organisational rules rather than by their judgement alone. Routine changes within agreed limits continue under delegated approval rules. Some changes require technical experts: those that bring new risks, affect security-sensitive functions or personal data, have unclear technical requirements or go beyond agreed limits. If later planning reveals new risks, missing requirements or contradictions, the change returns to Checkpoint A.

As mentioned earlier, the specification documents are complemented by a shared plan for each change in

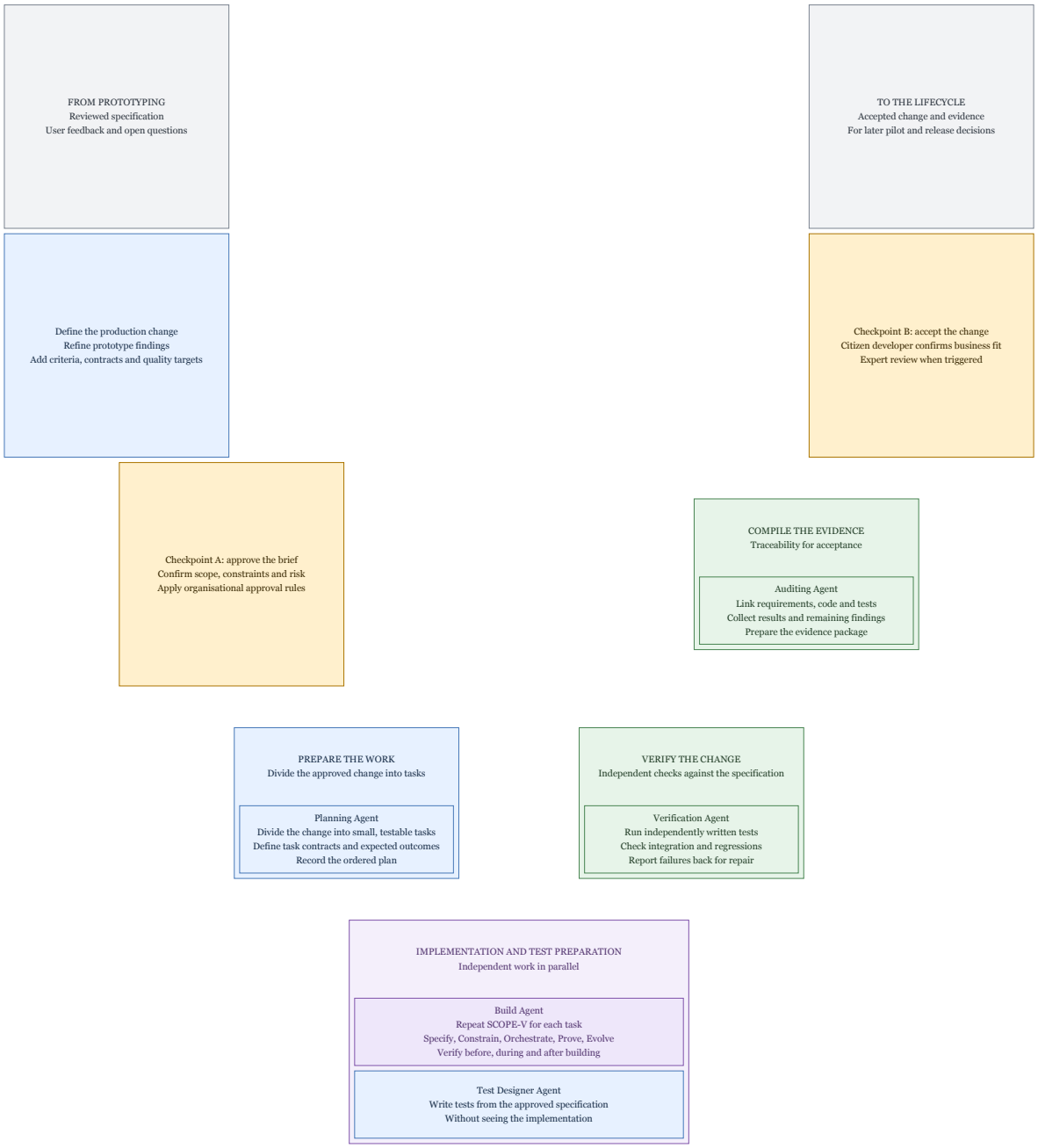


Figure 3. Own representation of an adapted Agile-V development cycle with SCOPE-V task execution (Koch, 2026; Koch & Wellbrock, 2026). Its integration into the stage-gate lifecycle and the distinction between change acceptance and production release are proposed in this thesis.

the agile V cycle. The plan, kept in a separate file such as plan.md, is created by a Planning Agent from the specification once it has been approved at Checkpoint A. The agent divides the approved change into small, testable tasks, links each task to its requirements and arranges them as an ordered checklist. For each task, it defines a task contract based on the approved specification, the expected observable outcome and the requirements that must be met before moving to the next task. The Planning Agent does not create the tests; the next agent turns these requirements into tests. It does not add new behaviour beyond the approved specification. Both humans and agents update the plan. After each task is implemented, they record relevant notes and implementation decisions and update the checklist to show progress. This follows the practice of an OpenAI engineering team that built a product entirely with coding agents. The team treated plans as first-class artefacts and captured larger pieces of work in execution plans with progress and decision logs, stored and versioned together with the code (Lopopolo, 2026). The accompanying guidance describes such plans as documents that are updated at every stopping point so that the work can be resumed from the plan alone (**friel2025**).

In this framework, agents record progress and implementation decisions in the plan rather than changing the approved specification. Changes to the specification must go through Checkpoint A. The plan also helps new sessions or agent tools continue the work, reducing the need for the citizen developer to explain it again. This corresponds to the persistent shared knowledge described in SDD, which different agents and tools can read and update (Díaz et al., 2026). If a handover to the IT department is needed because the work has become too complicated, the plan provides a record of progress and decisions, while version control preserves the history of project files.

The plan consists of two parts. The first part is written in plain language and contains the purpose of the change, the tasks with their task contracts and expected outcomes, the progress, the decisions taken and their reasons, unexpected findings and open questions. The second part contains technical notes for the Build Agent, such as the files and functions to be changed. Before the build starts, the citizen developer reviews the first part, which allows them to check the intended approach before a longer implementation begins (**friel2025**).

The plan in this framework differs from these practitioner sources in three ways (**friel2025**; Lopopolo, 2026). First, it is kept separate from the feature specification. In the original approach, the plan itself serves as the specification. Here, the specification is the approved contract and may only be changed through Checkpoint A, while the plan changes constantly. Second, the original approach instructs agents to resolve ambiguities on their own. In this framework, ambiguities that affect requirements are recorded as open questions and returned to the citizen developer or, where needed, to a specialist. Third, the plan is created by a dedicated Planning Agent, which organises the work but does not write code. This follows the distinction in SDD between a coordinating role and agents that carry out tasks in their own contexts (Díaz et al., 2026). In the original approach, one agent writes and maintains its own plan. Here, the plan is a shared working document: the Build, Verification and Auditing Agents record their progress, results and findings in it, and the citizen developer adds decisions and answers to open questions. Each agent writes only to the parts of the plan that its role allows. Unlike the coordinating agent in Agile V, which assigns work to the specialised agents (Koch & Wellbrock, 2026), the Planning Agent does not instruct the other agents; the reasons for this are explained below. These adaptations are design choices of this thesis. The documents and agents are explained below, and Table 1 gives an overview.

4.2.2.3.2 Bottom: building the change

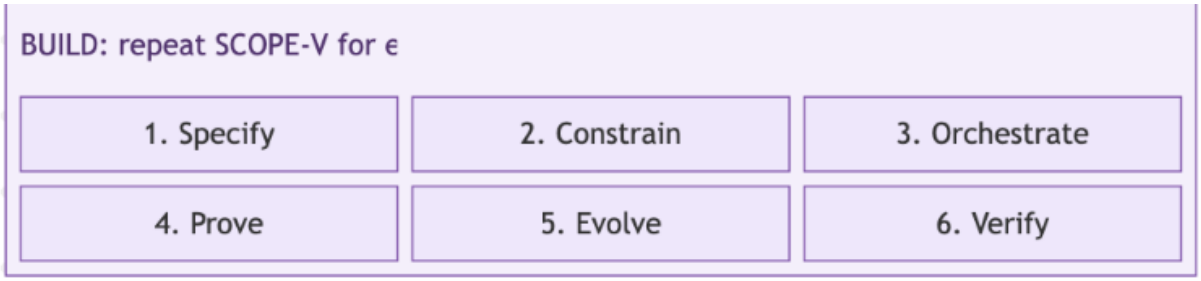
During the build, the plan is kept up to date after each task, either by the Planning Agent based on the reports of the Build Agent Agent, or directly by the Build Agents. When the change is complete, the Agents adds a short summary of the outcome and the lessons learned. Decisions recorded in the plan become part of the evidence bundle at Checkpoint B, and recurring lessons are moved into the system specification through the Evolve step. Completed plans are archived with the code, so that later changes can build on them. Both practitioner sources describe the work of experienced engineers and not citizen developers (**friel2025**; Lopopolo, 2026). Whether this approach is usable for citizen developers is therefore evaluated through eADR.

The Build Agent and the Test Designer Agent then do their work separately and in parallel (Koch & Wellbrock, 2026). The Build Agent carries out the tasks in the order set in the plan. The Test Designer Agent writes tests from the approved specification and from the task contracts and expected outcomes in the first part of the plan. It has no access to the generated code or to the technical part of the plan, since both contain implementation choices. As in the prototype stage, separate working contexts are used on

Table 1. Documents and agents in the adapted Agile V cycle (own representation).

Document	Purpose	Owner / timing
System specification (e.g. AGENTS.md)	Stable rules every change must respect, such as architecture, coding and data-protection rules	IT; read in every session
Feature specification	One change cycle: requirements, acceptance criteria, quality scenarios and contracts	Citizen developer; approved at Checkpoint A
Plan (plan.md)	Tasks, progress, decisions and open questions of one change	Created by the Planning Agent; updated by all agents and the citizen developer; from Checkpoint A to B
Evidence bundle	Test results, traceability matrix, scan results and a plain-language summary	Auditing Agent; before Checkpoint B
Agent	**Task**	**Does not**
Planning Agent	Splits the change into tasks and maintains the plan	Write code or add requirements
Build Agent	Builds each task through SCOPE-V	Change the independent tests
Test Designer Agent	Writes tests from the specification and task contracts	See the code
Verification Agent	Runs the independent tests	Fix code
Auditing Agent	Compiles the evidence bundle	Decide on acceptance

purpose: agents that check the work do not share the working memory of the Build Agent. Koch (2026) recommends separating building and testing agents when the risk is high. This thesis separates them for every change, because independent tests replace the routine code review that citizen developers cannot carry out. However, the separation cannot make the specification complete. If a requirement is wrong or missing, all agents may build on the same mistake. The quality of the specification at Checkpoint A therefore remains critical.



The Build Agent completes each task through the SCOPE-V cycle (Koch, 2026). In every session, it reads the system specification together with the feature specification and its current task in the plan. It carries out all six SCOPE-V steps for each task on its own:

Specify turns the task into a short brief with a goal, scope, acceptance criteria and required evidence.

Constrain sets limits, for example no changes to unrelated files and no new external libraries without a reason. Changes to security-sensitive code are flagged and handled according to the risk class of the change.

Orchestrate defines how the agent works: first examine the existing code, then plan, build in small steps and run quick checks.

Prove collects evidence that fits the risk. This can include unit tests (single parts), integration tests (parts working together), regression tests (existing functions still work), automated code analysis and security scans.

Evolve feeds confirmed lessons back into instructions, templates and tests, and removes outdated instructions. If the same correction is needed repeatedly, it is turned into a rule in the system specification after review, so that it applies to all future work (Díaz et al., 2026). Such changes to shared instructions or specifications follow the same change controls as other changes.

Verify is not a final step. Checks are repeated before, during and after building.

This follows Agentic Agile-V, where SCOPE-V describes how one agent works through a task (Koch, 2026). The same agent writes tests while building and runs quick checks itself. Within a single task, these self-checks are sufficient. Independence is ensured at the level of the whole change, where separate agents design and run the tests.

The other agents therefore work on the change as a whole, not on each task. The Test Designer Agent writes its tests once per change, based on the specification and the task contracts, while the Build Agent is building. When the Build Agent has finished all tasks, the Verification Agent runs these tests and the Auditing Agent then collects the evidence on the right side of the agile V framework. This matches the Agile V case study, where one cycle covered a whole piece of work with several requirements, not a single small task (Koch & Wellbrock, 2026).

This set-up has an advantage and a drawback. Only one agent works on each task, which keeps cost and effort low, and the independent checks happen where the change is accepted. However, mistakes that the Build Agent's own checks miss are only found after several tasks. For high-risk changes, the independent tests are therefore run after each task or after each milestone in the plan. This way of dividing the work between the agents is a design choice of this thesis. The test agent and the verification process will be explained in more detail in Section TBD, as it is one of the most critical steps in vibe coding (**bcgplatinion2026agentic**).

The agents do not talk to each other directly. Instead, they coordinate through shared, versioned files: the specification, the plan, the tests and the test reports. Each agent starts in a fresh session, reads only the files its role allows and writes its results back to these files. The order in which the agents work is fixed by the V-cycle (planning, then building and test design, then verification, then auditing). No agent decides this order. The citizen developer starts each step with a prompt that they have reviewed. The Planning Agent can suggest these prompts or check prompts the citizen developer has written. Each prompt mainly states which role the agent takes, which files it should read and what result it should produce. Because the prompts are concise, the citizen developer can understand them fully and review them quickly. They then check the outcome of each step before the next step begins.

The citizen developer therefore acts as the accountable owner of the change rather than as a programmer. They define the intent, approve the specification at Checkpoint A, answer open questions, adjust the plan where needed and accept the evidence at Checkpoint B. What they verify themselves is limited to what they can judge: functional behaviour and business rules. Non-functional qualities remain with automated checks, the testing agents and, where needed, IT specialists (see Section TBD).

This human-in-the-loop design is a deliberate choice. Vision papers propose fully automated development lifecycles in which an orchestrating meta-agent steers all phases, including verification, with little or no human involvement (**maes2026**). This thesis does not follow this direction, for three reasons. First, practitioner reports describe agents that complete roughly 70% of a task quickly but struggle with the remaining integration and edge cases, often ending in repeated fix loops (**osmani2024**; **rezun2026**). Second, studies of coding agents report constraint violations, unintended changes to the environment and false claims that a task is complete (**alhasan2026**; **uddin2026**). Third, without human involvement, no one in the organisation understands the application well enough to take responsibility for it, maintain it or hand it over (**mitchell2026**; Ginac, n.d.). For applications used in productive environments this last point is decisive. Accountability must stay with a person like the citizen developer, while technical accountability is delegated to specialists.

The same reasoning applies to how the agents themselves are coordinated. In Agile V, a coordinating agent assigns work to the specialised agents and tells them which files to read, so that each starts with a fresh context (Koch & Wellbrock, 2026). This thesis deliberately limits that role. The Planning Agent plans the change, keeps plan.md up to date and explains progress, decisions and problems to the citizen developer in non-technical language. However, it does not start or instruct the Build, Test Designer, Verification or Auditing Agents. There are four reasons for this. First, transparency: if agents manage other agents, several layers of abstraction separate the citizen developer from the work, which makes it harder for them to see what is happening and where the change stands. Second, independence: an agent that sees the code and also briefs the Test Designer could pass on implementation details. This would weaken the separation between building and testing, which replaces the code review that citizen developers cannot carry out. Third, traceability: when a person starts each step and the result is recorded in the shared files, every decision can be linked to a responsible person and to evidence. Fourth, collabo-

ration: the plan becomes the single place where the citizen developer follows and steers the work. The approved specification stays stable, so plan.md is the only document that changes continuously during the build. Every agent reports its results there in short, plain language: completed tasks, test results, audit findings, decisions and open questions. The citizen developer therefore does not need to follow several chat sessions, remember what each agent was told or rely on an agent's own claim that a task is complete. Such claims are not always reliable (**alhasan2026**). Instead, they read one document, keep it correct and react to its open questions. Each entry links to the underlying test or audit report, so progress can be checked against evidence rather than taken on trust. Because the plan is kept up to date at every stopping point, the work can also be resumed in a new session, or handed over to IT or other specialists, from this document alone (**friel2025**). The plan thus serves as a shared reference point not only between the citizen developer and the agents, but also between the citizen developer and the experts who support them.

Separate contexts remain important, because model performance tends to decline as the input grows longer and mixes unrelated information (**hong2025context**). The framework achieves this through a fresh session for each role rather than through an orchestrating agent. The Planning Agent may still use short-lived sub-agents for read-only tasks, such as summarising code or answering a question from the citizen developer. This keeps its own planning context free of technical detail (**anthropic2025context**). These sub-agents do not change any files. The trade-off is that the citizen developer has to carry out more steps manually. The eADR cycles evaluate whether this extra complexity is acceptable (Section TBD).

4.2.2.3.3 Right side: verifying and accepting the change

A separate Verification Agent, which corresponds to the Red Team Verifier in Agile V (Koch & Wellbrock, 2026), runs the independently written tests on the combined software. This differs from the Verify step of SCOPE-V. There, the Build Agent checks its own work; here, the Verification Agent tests the result independently. Tests on single tasks help find errors in individual changes. Integration and regression tests show whether the change affects other parts or breaks existing functions. If tests fail, the error reports are sent back to the Build Agent, which fixes the code and submits it again for testing. This feedback loop follows Agile V, where the Build Agent fixed the issues found by the verifier in a rework round (Koch & Wellbrock, 2026). It also follows the contract-based feedback loop of VibeContract (Wang, 2026). To avoid endless repair attempts, the number of automatic repair rounds is limited. If the change still fails after this limit, it is recorded in the plan as an unresolved finding and passed to an the citizen developer who can research the problem to solve it or pass it to an expert. The exact limit is design choice of the organisation.

An Auditing Agent, which corresponds to the Compliance Auditor in Agile V, then collects all evidence in one package. This package contains a traceability matrix, a table that links each requirement to the code and the tests that checked it (Koch & Wellbrock, 2026). It also contains the test results, the results of automated code analysis and security scans, important decisions and unresolved findings, as well as known remaining risks and, for production changes, a plan for undoing the change (Koch, 2026). The Auditing Agent also adds a short plain-language summary of this evidence to plan.md, so that the citizen developer can follow the result in the same document they use throughout the change.

At Checkpoint B, the change is accepted in two steps. First, the citizen developer decides whether the change meets the business need. This decision is based on the evidence package, not on reading the code. Second, a technical expert approves the change. Only throwaway prototypes are exempt from this technical approval. The risk class of a change does not decide whether a technical review takes place, but how deep it goes and which evidence it requires. If no expert is available, the change remains restricted or on hold and is not treated as verified.

This follows both Agile V sources, which keep technical approval in the loop. In Agile V, the responsible engineer holds final approval authority at the second human gate (**kochWellbrock2026AgileV**). **koch2026AgenticAgileV<empty citation>** defines four risk levels, each with stricter requirements for evidence and approval:

Table 2. Risk levels in Agentic Agile-V (Koch, 2026).

Risk level	Typical changes	Approval requirement
R0 exploratory	Throwaway prototypes and internal demos	Review optional
R1 routine	Documentation and small code clean-ups	Normal code review
R2 production	New features and bug fixes	Mandatory reviewer approval
R3 high assurance	Security-related functions, for example	Explicit sign-off and release gate

The prototype stage of this framework corresponds to R0. Most changes made by citizen developers are expected to fall under R2 and therefore require mandatory approval by a technical reviewer.

The framework does not remove this review but changes its form. Instead of reading every line of code, the expert checks whether the evidence package shows that the change meets its specification and looks more closely only where the evidence is weak. **diaz2026SDD<empty citation>** describe this shift as evidence-backed acceptance and argue that it makes review faster and more informative. The depth of the review is set explicitly for each type of change and can be revised as experience grows (**diaz2026SDD**). Which types of change belong to which risk level must be defined for SIXT HR rather than taken over directly from these sources. The split between business acceptance by the citizen developer and technical approval by the expert is a design choice of this thesis. It aims to keep technical control while reducing the effort that each change demands from scarce experts.

Errors in the code send the work back to the task cycle. Major changes to requirements or constraints send it back to Checkpoint A. For readability, the figure leaves out these return paths. An accepted change may be merged into the main code according to the repository rules, but acceptance does not mean release to production. Instead, the evidence from successive V-cycles feeds into the pilot and release decisions of the lifecycle, together with checks of the whole application and arrangements for operation. The same cycle is used again for changes during the pilot and in later operation.

The eADR evaluation will test whether this design works in practice. It examines whether citizen developers can write and check the required specifications, contracts and quality scenarios; whether the evidence package allows experts to review changes with manageable effort; whether expert availability becomes a bottleneck; and how many errors come from the specification rather than from the code.

4.2.2.3.4 Testing and verification

5 Evaluation and Findings

6 Discussion

7 Conclusion

References

Díaz, J., Gayoso, J., Cimminio, A., & Pérez, J. (2026). Spec-driven development for agentic software engineering: Harnessing human–agent teamwork. https://doi.org/10.48550/arXiv.2609.00252

Esnaola, L. M., Ramón, H. D., & Lanzarini, L. C. (2026). Can Generative AI Bridge the Gap? A Quasi-Experimental Study of Non-programmers with AI vs. Programmers without AI. *Empirical Software Engineering*, *31*, Article 75. https://doi.org/10.1007/s10664-026-10813-7

Farrag, S. E. (2026). The productivity-reliability paradox: Specification-driven governance for AI-augmented software development. https://doi.org/10.48550/arXiv.2605.01160

Feldman, M. Q., & Anderson, C. J. (2024). Non-Expert Programmers in the Generative AI Future. *Proceedings of the 3rd Annual Meeting of the Symposium on Human-Computer Interaction for Work*. https://doi.org/10.1145/3663384.3663393

Fetzer, D., Gimpel, H., Meindl, O., & Strickmann, J. (2026). Responsible engineering of information systems based on generative artificial intelligence: An action design research study at a german premium car manufacturer. *Business & Information Systems Engineering*, *68*(3), 583–608. https://doi.org/10.1007/s12599-025-00950-6

Fortes-Ferreira, M., Alam, M. S., & Bazilinskyy, P. (2025). Vibe Coding in Practice: Building a Driving Simulator Without Expert Programming Skills. *Adjunct Proceedings of the 17th International Conference on Automotive User Interfaces and Interactive Vehicular Applications*. https://doi.org/10.1145/3744335.3758482

François, P. A., Ciftci, S. A., Janiesch, C., & Plattfaut, R. (2025). Large language models for low-code process automation: Lowering the barriers for citizen developers [Paper 242]. *ACIS 2025 Proceedings*. https://aisel.aisnet.org/acis2025/242/

Gadde, A. (2025). Democratizing software engineering through generative AI and vibe coding: The evolution of no-code development. *Journal of Computer Science and Technology Studies*, *7*(4), 556–572. https://doi.org/10.32996/jcsts.2025.7.4.66

Galster, M., Mohsenimofidi, S., Lulla, J. L., Abubakar, M. A., Treude, C., & Baltes, S. (2026). Harness engineering for agentic AI coding tools: An exploratory study [Version 5, revised 30 June 2026]. https://doi.org/10.48550/arXiv.2602.14690

Ge, Y., Mei, L., Duan, Z., Li, T., Zheng, Y., Wang, Y., Wang, L., Yao, J., Liu, T., Cai, Y., Bi, B., Guo, F., Guo, J., Liu, S., & Cheng, X. (2025). A survey of vibe coding with large language models. https://doi.org/10.48550/arXiv.2510.12399

Ginac, F. (n.d.). *Epistemological Debt: Cognitive Atrophy and Systemic Collapse in AI-Dependent Software Engineering* [Research manuscript; publication year and venue not established from opening text]. https://drive.google.com/file/d/1tNXR0gOJaU9BK-kQWaS6zLgus_XunCly/view?usp=drivesdk

Hassan, A. E., Li, H., Lin, D., Adams, B., Chen, T.-H., Kashiwa, Y., & Qiu, D. (2025). Agentic software engineering: Foundational pillars and a research roadmap. https://doi.org/10.48550/arXiv.2509.06216

Hassan, A. E., Oliva, G. A., Lin, D., Chen, B., & Jiang, Z. M. ((2024). Towards AI-native software engineering (SE 3.0): A vision and a challenge roadmap. https://doi.org/10.48550/arXiv.2410.06107

Hoda, R. (2025). Toward agentic software engineering beyond code: Framing vision, values, and vocabulary [Also appears in ICSE-Companion 2026]. https://doi.org/10.48550/arXiv.2510.19692

Karpathy, A. (2025, February). There's a new kind of coding i call "vibe coding" [Accessed: 2026-09-17]. https://x.com/karpathy/status/1886192184808149383

Kobiella, C., Breidenstein, D., & Schmidt, A. (2026). From throw-away to takeaway: How GenAI and vibe coding accelerate prototyping across technical skill levels. *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*, 1–23. https://doi.org/10.1145/3772318.3790757

Koch, C. (2026). Agentic Agile-V: From Vibe Coding to Verified Engineering in Software and Hardware Development [Stored arXiv version 1]. https://arxiv.org/abs/2605.20456v1

Koch, C., & Wellbrock, J. A. (2026). Agile V: A Compliance-Oriented Framework for AI-Augmented Engineering — From Concept to Audit-Ready Delivery [Stored arXiv version 1]. https://arxiv.org/abs/2602.20684v1

Li, H., Zhang, H., & Hassan, A. E. (2025). The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering [Stored arXiv version 1]. https://arxiv.org/abs/2507.15003v1

Lopopolo, R. (2026, February 11). *Harness engineering: Leveraging Codex in an agent-first world*. OpenAI. Retrieved October 22, 2026, from https://openai.com/index/harness-engineering/

Mandl, P., & Mandl, P. (2026). AI-driven software development: A pragmatic path to agentic development processes. https://doi.org/10.48550/arXiv.2606.15283

Meske, C., Hermanns, T., von der Weiden, E., Loser, K.-U., & Berger, T. (2025). Vibe coding as a reconfiguration of intent mediation in software development: Definition, implications, and research agenda. *IEEE Access*, *13*, 213242–213259. https://doi.org/10.1109/ACCESS.2025.3645466

Sapkota, R., Roumeliotis, K. I., & Karkee, M. (2025). Vibe coding vs. agentic coding: Fundamentals and practical implications of agentic AI. https://doi.org/10.48550/arXiv.2505.19443

Sarkar, A., & Drosos, I. (2025). Vibe coding: Programming through conversation with artificial intelligence. *Proceedings of the 36th Annual Conference of the Psychology of Programming Interest Group*. https://www.microsoft.com/en-us/research/publication/vibe-coding-programming-through-conversation-with-artificial-intelligence/

StrongDM AI. (2026, February 6). *Software factories and the agentic moment*. StrongDM. Retrieved October 23, 2026, from https://factory.strongdm.ai/

Wang, S. (2026). VibeContract: The Missing Quality Assurance Piece in Vibe Coding [Stored arXiv version 1; Placeholder 2024 date, conference name and DOI omitted]. https://arxiv.org/abs/2603.15691v1